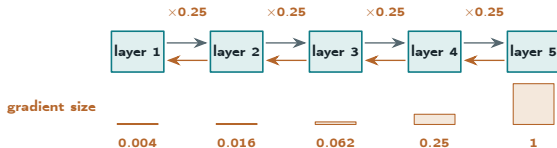


L06 The representation catalog, and learning on it

*Which 3D representation to keep depends on the questions the robot will ask of it;
which network to use depends on the representation.*

Vanishing and exploding gradients

finishing Wednesday: training, then convolutional networks



- The gradient reaching an early layer is a product of one factor for every layer after it.
- Factors below 1 shrink it: **vanishing gradients**, and early layers stop learning. With sigmoid slopes of at most 0.25, ten layers give $0.25^{10} \approx 10^{-6}$.
- Factors above 1 blow it up: **exploding gradients**, often helped along by unbounded activations, and the loss jumps to NaN.
- **Dead ReLU**: a unit whose sum is negative for every input outputs 0 with slope 0, and never updates again.

Keeping training stable

- **ReLU or a variant** in hidden layers. Leaky ReLU keeps a small slope below zero, so its units cannot die.
- **A lower learning rate:** large steps are what push ReLU units dead and gradients up.
- **Sensible starting weights**, random and scaled to the number of inputs of each layer.
- **Normalisation** of the input data, and of the outputs of intermediate layers.

Activation, learning rate and normalisation are chosen together. Change one and the best setting of the others moves.

Normalisation

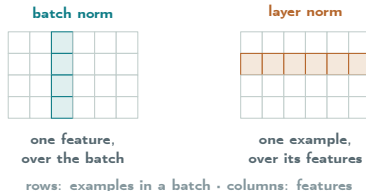
■ Batch normalisation of one feature over a batch of B examples

$$\mu = \frac{1}{B} \sum_{j=1}^B z_j, \quad \sigma^2 = \frac{1}{B} \sum_{j=1}^B (z_j - \mu)^2$$

$$\hat{z}_j = \frac{z_j - \mu}{\sqrt{\sigma^2 + \epsilon}}, \quad \text{output}_j = \gamma \hat{z}_j + \beta$$

$\epsilon > 0$ is tiny, so nothing divides by zero. γ and β are learned.

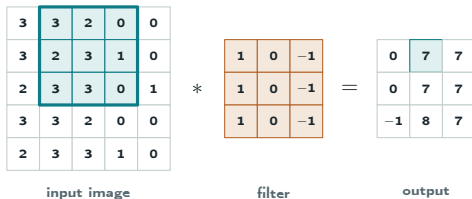
Normalisation rescales numbers to control their mean and spread. For input images: subtract the dataset mean and divide by its standard deviation, per colour channel.



Batch norm or layer norm

- **Batch norm** averages each feature over the batch. At test time there may be one image, so it uses running averages saved during training. Forgetting `model.eval()` is a classic bug.
- It degrades at small batch sizes, and 3D perception often trains with batches of 2 to 8.
- **Layer norm** averages over the features of one example, so it behaves the same at any batch size, including 1. Transformers use it.

Convolution: slide a small filter



$$(I * K)(r, c) = \sum_m \sum_n I(r + m, c + n) K(m, n)$$

- Lay the filter on a patch, multiply entry by entry, add. The shaded output is $(3 + 2 + 3) - (0 + 1 + 0) = 7$: the patch's left column minus its right. Slide one pixel and repeat.
- The same nine weights are used at every position: **weight sharing**. An edge detector works anywhere in the image.

Filters you have already met

1	2	1
2	4	2
1	2	1

blur
× 1/16

-1	0	1
-2	0	2
-1	0	1

vertical edges
Sobel

w_{11}	w_{12}	w_{13}
w_{21}	w_{22}	w_{23}
w_{31}	w_{32}	w_{33}

learned
set by backprop

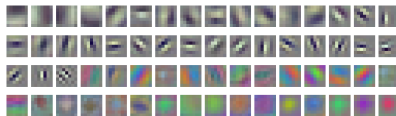
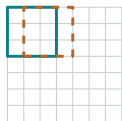


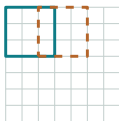
Figure: Olah et al., Distill (2020), CC BY 4.0

- L04's image pyramid blurred with a Gaussian, $G(x, y) = \frac{1}{2\pi\sigma^2} e^{-(x^2+y^2)/2\sigma^2}$, sampled on a small grid. A bigger filter or a larger σ blurs more.
- An edge filter responds where brightness changes from one side to the other.
- SIFT and ORB were built from filters like these, designed by hand. A convolutional network treats the nine numbers as weights and learns them. On the right, the first-layer filters of a trained network: edges and colour blobs.

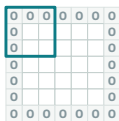
Stride and padding



stride 1
output 5 wide



stride 2
output 3 wide



padding 1
5 wide in, 5 wide out

- **Stride:** how far the filter moves each time. Larger strides give smaller outputs; above 2 is rare.
- **Padding:** a border of zeros around the input, to control the output size.

■ Output width for input W , filter K , padding P , stride S

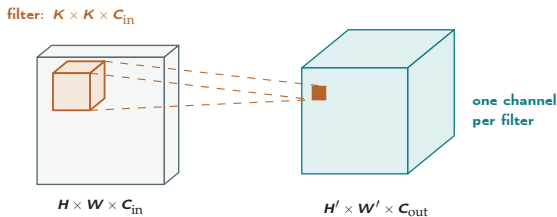
$$\text{out} = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$

$W = 7, K = 3, P = 0$: stride 1 gives 5, stride 2 gives 3.

$W = 224, K = 3, P = 1$: stride 1 gives 224, stride 2 gives 112.

$P = (K - 1)/2$ with $S = 1$ keeps the size unchanged.

Channels: many filters at once



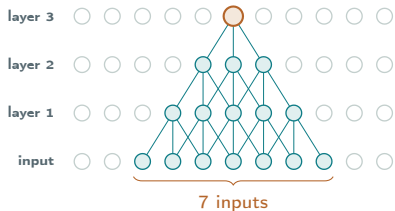
- A colour image has 3 **channels**. A filter spans all of them: $3 \times 3 \times 3$ weights and a bias.
- Use C_{out} filters and the output has C_{out} channels, one **feature map** per filter. This is the layer's **depth**.

■ **Parameters:** 3×3 filters, 3 channels in, 64 out

$$3 \cdot 3 \cdot 3 \cdot 64 + 64 = 1,792$$

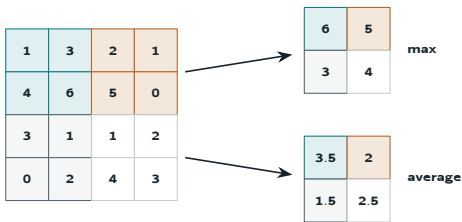
The count does not depend on the image size.

Receptive field



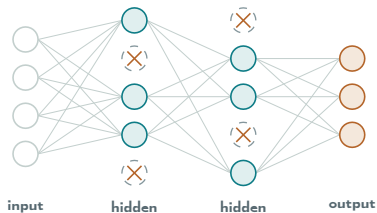
- The **receptive field** of a neuron is the patch of the input it depends on.
- Each 3×3 layer widens it by 2, so three layers see 7×7 . Stride and pooling widen it faster.
- How large it must be depends on the task. Fine geometry needs small neighbourhoods; judging motion or the layout of a room needs large ones.

Pooling



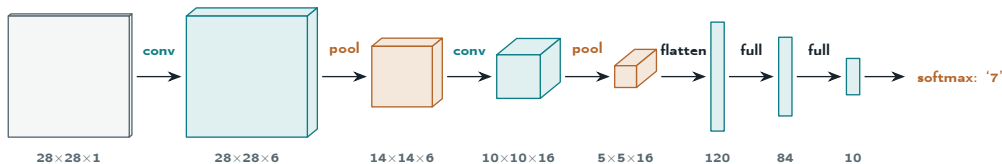
- **Max pooling** keeps the largest value in each window; **average pooling** keeps the mean.
- A 2×2 window with stride 2 halves height and width, and has no weights to learn.
- It downsamples the feature maps between layers: less computation later, and a wider receptive field.

Dropout



- During training, zero each neuron's output with probability p , often 0.5, and draw a new random set at every step.
- No neuron can rely on one particular partner being there, which reduces overfitting.
- At test time every neuron is used, with outputs scaled to match what the next layer saw in training.

A convolutional network



- Convolutions and pooling shrink height and width and add channels: more kinds of feature, each summarising a wider area.
- Then flatten, finish with fully connected layers, and softmax over the classes.
- This is LeNet (LeCun et al., 1998), which read handwritten digits. AlexNet (2012) used the same recipe, far larger, on GPUs, and won ImageNet.

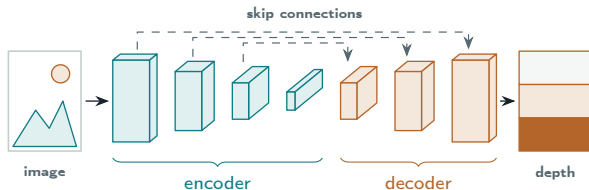
What the layers learn



Figure: Olah, Mordvintsev & Schubert, Distill (2017), CC BY 4.0

- Early filters respond to edges and colour; later layers to textures, patterns, parts, and whole objects.
- A trained CNN has learned its own image pyramid and its own feature descriptors, tuned to its task.

From a classifier to a depth network



Drop the fully connected layers. An **encoder** shrinks the image into features, a **decoder** grows them back to one output per pixel, and **skip connections** carry fine detail across (U-Net). Train it with a loss at every pixel against measured depth.

The loss decides what a network can learn. MiDaS's loss ignores scale and shift, so its depth is relative.

What you should be able to do

L05, deep learning

- Run one gradient-descent step on $\hat{y} = wx + b$ by hand, chain rule included.
- Say why a network needs a nonlinear activation, and pick one for the hidden layers and one for the output.
- Turn scores into probabilities with softmax, and score them with cross-entropy.
- Read a pair of loss curves and decide whether to keep training.
- Compute the output size and the parameter count of a convolutional layer.

To go further: 3Blue1Brown, *Neural networks*, chapters 1–4; *Dive into Deep Learning* (d2l.ai), chapters 3–5 and 7; the Stanford CS231n notes on convolutional networks.

Representing and reconstructing 3D

L06 · Unit 1 begins

What a robot can do with a 3D map depends on the representation it keeps.

Unit 0 is done. Five lectures ago a camera was a matrix; now you can write down where a robot is, what its sensors measured, and how a network is fitted to examples. Unit 1 is about what to build from those measurements.

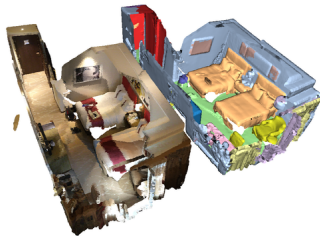


Figure: Minkowski Engine documentation, MIT License

Today

1. **What the robot asks of a map:** the questions that choose the representation.
2. **The catalog:** point clouds, voxels and octrees, signed distance, meshes, implicit functions, 3D Gaussians.
3. **Networks on point sets:** PointNet, and why a function of a set must ignore the order of its points.
4. **PointNet++:** neighbourhoods and a hierarchy, as a CNN has.
5. **Sparse convolution:** MinkowskiNet, and convolution that skips empty space.

Reading: PointNet (today's response), PointNet++ and MinkowskiNet. The catalog is a survey: TSDF fusion, the ESDF and neural fields come back on Wednesday, Gaussians in L09.

What does the robot ask of its map?

Task	Question put to the map	What the map must hold
Avoid collisions	Is this sphere free? How far is the nearest obstacle?	free space, and distance
Localise	Where does this new scan fit?	geometry to align against
Grasp	Where is the surface, and which way does it face?	surface and normals, at millimetres
Predict a view	What would the camera see from here?	appearance
Explore	What have I not seen yet?	unknown kept apart from free
Run for hours	How large has the map grown?	memory that grows with the scene, not with time

Most systems keep more than one representation, each answering a different question.

Point clouds

- A **point cloud** is a set of N points (x, y, z) , often with a colour, an intensity or a normal. It is what LiDAR and depth cameras return.
- It has no connectivity, no surface, and no record of free space: an empty region and an unobserved one look the same.
- **Normals** come from neighbours: the direction in which the k nearest points spread least, the eigenvector of their covariance with the smallest eigenvalue. A **k-d tree** finds neighbours in about $O(\log N)$ per query.

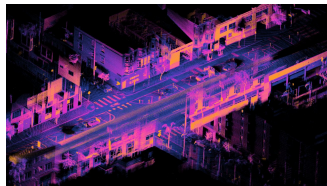
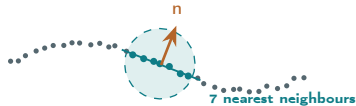


Image: Daniel L. Lu, CC BY 4.0, via Wikimedia Commons



A depth image is a point cloud on a grid

Back-project pixel (u, v) with its measured depth Z :

$$X = \frac{(u - c_x) Z}{f_x}, \quad Y = \frac{(v - c_y) Z}{f_y}$$

That is $[X \ Y \ Z]^T = Z K^{-1} [u \ v \ 1]^T$. With $f = 525$ and $(c_x, c_y) = (319.5, 239.5)$, pixel $(420, 300)$ at 1.5 m is at $(0.287, 0.173, 1.5)$ m.

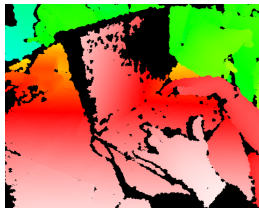
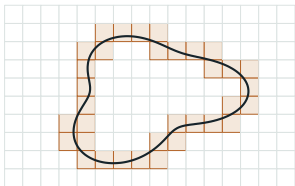


Image: Kolossos, CC BY-SA 3.0, via Wikimedia Commons

- An **organised** cloud keeps the $H \times W$ grid, so pixel neighbours are point neighbours; an **unorganised** cloud is a list.
- A 640×480 frame is 307,200 points; at 30 Hz, 9.2 million points a second.

Voxel grids



34 of 160 cells occupied; all 160 stored

- A **voxel** is a cell of a regular 3D grid. It stores occupancy, a distance, or a colour.
- Asking what is at (x, y, z) is an array index, and neighbours are adjacent cells, so 3D convolution works directly.

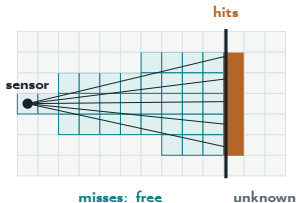
■ Memory for a $10 \times 10 \times 3$ m room, one 4-byte float per voxel

5 cm : $200 \cdot 200 \cdot 60 = 2,400,000$ voxels = 9.6 MB

1 cm : $1000 \cdot 1000 \cdot 300 = 3 \times 10^8 = 1.2$ GB

Five times finer costs $5^3 = 125$ times the memory, and most of those voxels are empty.

Occupancy is a probability



- A beam is evidence: the cells it crosses are probably free, the cell where it ends probably occupied.
- In **log-odds** $l = \log \frac{p}{1-p}$, Bayes' rule for independent measurements is a sum.

■ Update one cell with $p_{\text{hit}} = 0.7$, $p_{\text{miss}} = 0.4$, starting from $p = 0.5$

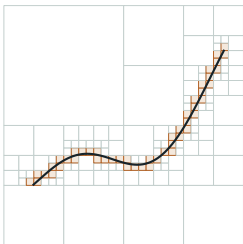
$$\text{hit: } l \leftarrow l + \log \frac{0.7}{0.3} = l + 0.85$$

$$\text{miss: } l \leftarrow l + \log \frac{0.4}{0.6} = l - 0.41$$

Three hits: $l = 2.54$, $p = 1/(1 + e^{-l}) = 0.93$.
Then one miss: $l = 2.14$, $p = 0.89$.

Clamp l to $[-2.0, 3.5]$ (p from 0.12 to 0.97), so a cell can change its mind when a door opens.

Octrees



148 leaves instead of 1024 cells

- Split a cube into eight only where something is. A large empty or unknown region stays a single node.
- **OctoMap** keeps log-odds occupancy in an octree and stores unknown separately from free, which exploration needs (L08).
- d levels of leaves of size r span $r \cdot 2^d$: 16 levels of 1 cm reach 655 m. A lookup costs $O(d)$.

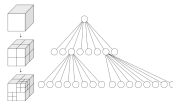
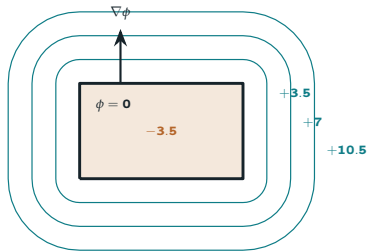


Image: WhiteTimberwolf, CC BY-SA 3.0, via Wikimedia Commons

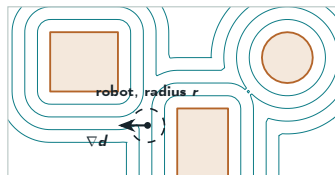
Signed distance



$$\phi(x) = \pm \min_{s \in \text{surface}} \|x - s\|$$

- Positive in free space, negative inside, zero on the surface: the surface is the level set $\phi = 0$.
- $|\nabla\phi| = 1$ almost everywhere, and $\nabla\phi$ on the surface is the normal.
- Stored on a grid, the surface falls between samples where the sign changes, so it is located to a fraction of a voxel.

TSDF and ESDF



contours of distance to the nearest obstacle

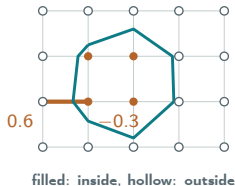
- **TSDF**: signed distance along each camera ray, clamped at $+\tau$ in front of the surface and not updated more than τ behind it, where the camera saw nothing.
- Averaging many of these in each voxel fuses depth frames (Wednesday).
- **ESDF**: the distance to the nearest obstacle, in every voxel. A robot of radius r at x is free if $d(x) > r$: one lookup.
- ∇d pushes a path away from obstacles. Voxblox builds the ESDF from the TSDF.

Meshes, and getting one from a grid

- A **mesh** is vertices and triangles: an explicit surface with connectivity, what renderers, physics engines and grasp planners take.
- It has no free space and is awkward to update, so maps are fused in a grid and meshed afterwards.
- **Marching cubes**: the signs at a cube's 8 corners give 256 cases, 15 up to symmetry, each a few triangles.



Image: Trevorgoodchild, public domain, via Wikimedia Commons



Where the surface cuts an edge with values ϕ_a and ϕ_b :

$$t = \frac{\phi_a}{\phi_a - \phi_b} = \frac{0.6}{0.6 - (-0.3)} = 0.67$$

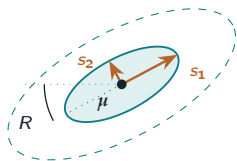
Implicit functions: a network as the map



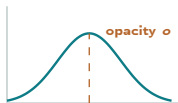
- Replace the grid with a network $f_\theta(x, y, z)$ that returns occupancy (Occupancy Networks) or signed distance (DeepSDF).
- No grid fixes the resolution; the memory is the weights.
- Every query is a forward pass, and nothing guarantees a sensible answer far from the training data.
- **NeRF** returns colour and density instead, and is the second half of Wednesday.

3D Gaussians

1 of 5: one Gaussian



solid: one standard deviation; dashed: two



light blocked, along the long axis

$$G(x) = \exp \left(-\frac{1}{2}(x - \mu)^\top \Sigma^{-1}(x - \mu) \right)$$

- **Mean μ :** where it sits.
- **Covariance Σ :** its shape and orientation, stored as a rotation R and three scales s_1, s_2, s_3 .
- **Opacity o :** how much light it blocks at its centre; at x it blocks $o G(x)$.
- **Colour:** three numbers, or more so it can change with viewing direction.

Next time

L07 · the rest of this, then registration and maps

Wednesday picks up here: the rest of the Gaussians, the catalog side by side, and the networks that read point sets, from PointNet to sparse convolution.

Then registration, which is what a robot does before it can fuse anything: ICP and its variants, and two ways to hold a map, volumetric and neural.

Response due Wednesday: Voxblox. Also assigned: NeRF, Instant-NGP and NICE-SLAM. **Both diagnostic quizzes are due Sun Sep 27.**